

Design & Analysis of Algorithms: An Introduction

Dr. Muhammad Sarwar Ehsan
adean.foit@ucp.edu.pk

Department of Computer Science,
University Central of Punjab

September 29, 2025

OUTLINE

1 INTRODUCTION

- Course Outcomes

2 COURSE OUTLINE

- Books

3 SCHEDULE

4 GRADING POLICY

5 WHAT IS AN ALGORITHM?

6 ANALYSIS OF ALGORITHMS

INTRODUCTION

INSTRUCTOR

- **Name:** Prof. Dr. Muhammad Sarwar Ehsan
- **Professor / Associate Dean**
- **email:** adean.foit@ucp.edu.pk
- **Office:** Building A (F-201 Cabin # 03)
- Teaching (Undergrad): DAA, Probability and Statistics, Signals & Systems, Digital Signal Processing
- Teaching (Grad): Stochastic Processes, Advanced Digital Signal processing, Multimedia Systems
- Research Interests: Multimedia Processing & Artificial Intelligence, Information Theory

COURSE

CSAL 3233: Design & Analysis of Algorithms

COURSE LEARNING OUTCOMES

- CLO1:** Analyze the time complexity of different algorithms using asymptotic notation. [C4- Synthesize]
- CLO2:** Apply different strategies (brute-force, greedy, divide and conquer, and dynamic programming) to solve a computing problem . [C3-Application]
- CLO3:** Solve problems using graph algorithms, including single-source, Spanning Tree, and Graph traversing algorithms. [C3-Application]

STUDYING AT THE UNIVERSITY

- You have an opportunity to learn on the lectures. I can not force you to learn.
- You have more responsibility on your learning than you had in vocational school or senior high school.
- 1 cr $\approx$ 30 hours of work. 3 cr = 90 hours of work. You will spend 45 hours on the lectures.
- Which means that you should use about 45 hours of your own time for studying!
- If I proceed too fast or too slow, please interject me (or tell me by email).
- Do not hesitate to ask.

COURSE OUTLINE

- Introduction to the course and course objectives
- What is an algorithm? Fundamental of the algorithmic problem solving, important problem types fundamental data structures
- Fundamentals of the analysis of algorithm efficiency
- Brute force and exhaustive search
- Divide-and-Conquer
- Dynamic Programming
- Greedy Techniques
- Graph algorithms
- Different complexity classes (P , NP , NP -complete)

TEXT & REFERENCE BOOKS

TEXT BOOK

T. H. Corman, C. E. Leiserson, R. . Rivest and C. Stein,
Introduction to Algorithms. 4th Edition, MIT Press, 2009.

REFERENCE BOOKS

- M. T. Goodrich, and R. Tamassia: *Algorithm Design and Applications*. Wiley Inc., 2015.
- R. Sedgewick and K. Wayne: *Algorithms*. Addison-Wesley, 4th edition, 2011.
- S. Dasgupta, C. Papadimitriou and U. Vazirani: *Algorithms*. McGraw-Hill, 2006.

LECTURE SCHEDULE

Section E1

Tuesday	12:00 - 13:55	A - 112
Thursday	16:00 - 16:55	A - 105

Section E7

Monday	13:00 - 14:55	A - 103
Thursday	13:00 - 13:55	A - 117

- **Office Hours (Building A F201 cabin # 3)**

- ▶ Monday 11:00 - 12:00
- ▶ Tuesday 10:00 - 11:00
- ▶ Wednesday 12:00 - 13:00
- ▶ Thursday 11:00 - 12:00

GRADING POLICY

- **Quizzes:** 15%
- **Assignments:** 10%
- **Class Participation:** (5%)
- **Mid Term:** 20%
- **Final:** 50%

IMPORTANT NOTE

- No body will be allowed to enter the class after the arrival of the instructor
- **No relaxation in attendance**, once it is marked in the class
- There will be a participatory work in every class
- Dates for quizzes are tentative. There will be **surprise quizzes**

WHAT IS AN ALGORITHM?

- Algorithm
 - ▶ Named after, **Muhammad ibn Musa al-Khwarizmi**
- Definition
 - ▶ An algorithm is a definite procedure for solving a problem in finite number of steps
 - ▶ Algorithm is a well defined computational procedure that takes some value (s) as input, and produces some value (s) as output.
 - ▶ Algorithm is finite number of computational statements that transform input into the output

PROPERTIES OF ALGORITHMS

- Has input values zero or more inputs
- Produces output values at least one output
- Definiteness clear, precise, and unambiguous
- Finiteness terminate after a finite number of steps
- Effectiveness Each individual step must be basic, feasible, and doable in finite time
- Generality general class of problems

EUCLID'S ALGORITHM FOR GREATEST COMMON DIVISOR

FIRST ALGORITHM

```
1:  $A \leftarrow$  First integer
2:  $B \leftarrow$  Second integer
3: while ( $B \neq 0$ ) do
4:   while ( $A > B$ ) do
5:      $A \leftarrow A - B$ 
6:   end while
7:    $B \leftarrow B - A$ 
8: end while
9: return  $A$ 
```

SECOND ALGORITHM

```
1:  $A \leftarrow$  First integer
2:  $B \leftarrow$  Second integer
3: while (1) do
4:    $REM \leftarrow A \% B$ 
5:   if ( $REM == 0$ ) then
6:     Break
7:   end if
8:    $A \leftarrow B$ 
9:    $B \leftarrow REM$ 
10: end while
11: return  $B$ 
```

ALGORITHM – EXAMPLES

- The rules of how to play a game
- Directions for driving from A to B
- Finding the largest number in a list of numbers
- Finding greatest common divisor between two numbers
- Matrix Multiplication
- **The Traveling Salesman Problem**
- **Clique Problem**
- **Knapsack Problem**

WHAT KINDS OF PROBLEMS ARE SOLVED BY ALGORITHMS?

- The Human Genome Project
- The internet quickly access and retrieve large amounts of information
- Electronic commerce
- Resource allocation
- Shortest path search
- Data compression
- Machine learning

IMPORTANT DESIGNING TECHNIQUES

BRUTE FORCE: Straightforward, naive approach—Mostly expensive

DIVIDE-AND-CONQUER: Divide into smaller sub-problems, e.g., merge sort

GREEDY APPROACH: Select the best option in a given scenario

DYNAMIC PROGRAMMING: Programming—Dependent sub-problems, reuse results

ANALYSIS OF ALGORITHMS

- Two essential approaches to measuring algorithm efficiency:
- Empirical analysis:
 - ▶ Program the algorithm and measure its running time on example instances
- Theoretical analysis
 - ▶ Employ mathematical techniques to derive a function which relates the running time to the size of instance
- **In this course our focus will be on Theoretical Analysis**

ANALYSIS OF ALGORITHMS

WORST CASE: The worst outcome for any possible input. We often concentrate on this for our analysis as it provides a clear upper bound of resources and an absolute guarantee.

AVERAGE CASE: Measures performance over the entire set of possible instances.

- Very useful, but treat with care: what is “average”?
- Random (equally likely) inputs vs. real-life inputs

BEST CASE: The best outcome for any possible input.

- provides lower bound of resources

EMPIRICAL ANALYSIS

- Write a program implementing the algorithm
- Run the program with inputs of varying size and compositions
- Use timing routines to get an accurate measure of the actual running time, e.g., `System.currentTimeMillis()`
- Implementation dependent
- Platform dependent
- Data dependent

THEORETICAL ANALYSIS

- Data independent
- Platform independent
- Language independent
- Implementation independent
- Characterizes running time as a function of input size, n .
Easy to extrapolate without risk

WHY ALGORITHM ANALYSIS IS IMPORTANT?

- Algorithm analysis is an important part of computational complexity theory, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem
- To predict the behavior of an algorithm without implementing it on a specific computer
- It is much more convenient to have simple measures for the efficiency of an algorithm than to implement the algorithm and test the efficiency every time a certain parameter in the underlying computer system changes
- It is impossible to predict the exact behavior of an algorithm. There are too many influencing factors
- The analysis is thus only an approximation; it is not perfect
- More importantly, by analyzing different algorithms, we can compare them to determine the best one for our purpose

WHAT DO WE ANALYZE ABOUT ALGORITHMS?

- Algorithms are analyzed to understand their behavior and to improve them if possible
- Correctness
 - ▶ Does the input/output relation match algorithm requirement?
- Amount of work done
 - ▶ Basic operations to do task
- Amount of space used
 - ▶ Memory used
- Simplicity, clarity
 - ▶ Verification and implementation
- Optimality
 - ▶ Is it impossible to do better?

PROBLEM SOLVING PROCESS

- Problem
- Strategy
- Algorithm
 - ▶ Input
 - ▶ Output
 - ▶ Steps
- Analysis
 - ▶ Correctness
 - ▶ Time & space optimality
- Implementation
- Verification

COMPUTATION MODEL FOR ANALYSIS

- To analyze an algorithm is to determine the amount of resources necessary to execute it. These resources include computational time, memory and communication bandwidth
- Analysis of the algorithm is performed with respect to a computational model called **RAM (Random-Access Machine)**
- A RAM is an idealized uni-processor machine with an infinite large random-access memory
 - ▶ Instruction are executed one by one
 - ▶ All memory equally expensive to access
 - ▶ No concurrent operations
 - ▶ Constant word size
 - ▶ All basic operations take unit time
 - ▶ Does not account for the memory hierarchy such as cache or virtual memory

COMPLEXITY OF AN ALGORITHM

- The complexity of an algorithm is the amount of work the algorithm performs to complete its task. It is the level of difficulty in solving mathematically posed problems as measured by:
 - ▶ Time (time complexity)
 - ▶ No. of steps or arithmetic operations (computational complexity)
 - ▶ Memory space required (space complexity)
- Complexity is a function $T(n)$ which yields the time (or space) required to execute the algorithm of a problem of size “ n ”

PSEUDO-CODE

- High-level description of an algorithm
- More structured than English prose but Less detailed than a program
- Preferred notation for describing algorithms
- Hides program design issues

EXAMPLE

```
1:  $i \leftarrow 10$ 
2: if  $i \geq 5$  then
3:    $i \leftarrow i - 1$ 
4: else
5:   if  $i \leq 3$  then
6:      $i \leftarrow i + 2$ 
7:   end if
8: end if
```

ELEMENTARY OPERATION

- An elementary operation is an operation which takes constant time regardless of problem size
- Example of elementary operations include
 - ▶ variable assignment
 - ▶ arithmetic operations $(+, -, \times, /)$ on integer
 - ▶ comparison operations $(a < b)$
 - ▶ boolean operation
 - ▶ accessing an element of an array
- We will measure number of steps taken in term of size of input

COMPONENTS OF AN ALGORITHM

- Variables and values
- Instructions
- Sequences
- Selections
- Repetitions

INSTRUCTION AND SEQUENCE

- A linear sequence of elementary operations is also performed in constant time
- More generally, given two program fragments P_1 and P_2 which run sequentially in times t_1 and t_2
 - ▶ use the maximum rule which states that the larger time dominates
 - ▶ complexity will be $\max(t_1, t_2)$

EXAMPLE

1: $x \leftarrow a$	▷ 1
2: $x \leftarrow a + b * c/h - u$	▷ 1
3: $a > b$	▷ 1

$$T(n) = 1 + 1 + 1 = 3$$

SELECTION

- If $\langle test \rangle$ then P_1 else P_2 structures are a little harder; conditional loops
- The maximum rule can be applied here too: $\max(t_1, t_2)$, assuming t_1, t_2 are times of P_1, P_2

EXAMPLE

```
1: if  $i \geq maxval$  then  
2:    $i \leftarrow 0$   
3: else  
4:   if  $i + k \leq maxval$  then  
5:      $i \leftarrow i + k$   
6:   end if  
7: end if
```

$$T(n) = 1 + \max(2, 1) = 3$$

REPETITION (LOOPS)

- Analyzing loops: Any loop has two parts:
 - How many iterations are performed?
 - How many steps per iteration?
- Running time of a for-loop is at most the running time of the statements inside the for-loop times number of iterations
- This approach is reasonable, provided n is positive
- For nested loops time is mn

EXAMPLE

```
1: for <1 to n> do
2:   <do stuff>
3: end for
```

$$T(n) = nt$$

ANALYSIS EXAMPLE

EXAMPLE

1: $n \leftarrow 0$	$\triangleright 1$
2: $sum \leftarrow 0$	$\triangleright 1$
3: $i \leftarrow 0$	$\triangleright 1$
4: while $i < n$ do	$\triangleright n$
5: $number \leftarrow$ read input from user	$\triangleright n - 1$
6: $sum \leftarrow sum + number$	$\triangleright n - 1$
7: $i \leftarrow i + 1$	$\triangleright n - 1$
8: end while	
9: $mean \leftarrow sum/n$	$\triangleright 1$

The computing time for this algorithm is

$$T(n) = 1 + 1 + 1 + n + (n - 1) + (n - 1) + (n - 1) + 1 = 4n + 1$$

ASYMPTOTIC GROWTH RATE

- Changing the hardware/software environment
 - ▶ Affects $T(n)$ by constant factor, but does not alter the growth rate of $T(n)$
- The growth of the complexity functions is what is more important for the analysis and is a suitable measure for the comparison of algorithms with increasing input size n
- Asymptotic notations are used to compute the complexity because different implementations of algorithm may differ in efficiency
- Helps to determine best-case, worst-case and average case behavior

ASYMPTOTIC GROWTH RATE

- Two reasons why we are interested in asymptotic growth rates
- **Practical purposes:** For large problems, when we expect to have big computational requirements
- **Theoretical purposes:** concentrating on growth rates frees us from some important issues
 - ▶ **fixed costs** (e.g. switching the computer on!), which may dominate for a small problem size but be largely irrelevant
 - ▶ **machine and implementation details**
 - ▶ The growth rate will be a compact and **easy to understand the function**

EXAMPLE: LINEAR SEARCH ALGORITHM

- Given a linear array A containing n elements, locate the position of an Item ' x '
- Comparing ' x ', one by one, with each element in A .

1: LinearSearch (A, x)

2: $i \leftarrow 1$ $\triangleright 1$

3: $n \leftarrow \text{SizeofArray}$ $\triangleright 1$

4: **while** ($i \leq n$) **do** $\triangleright n + 1$

5: **if** ($A[i] == x$) **then** $\triangleright n$

6: **return** TRUE $\triangleright 1$

7: **else**

8: $i \leftarrow i + 1$ $\triangleright n - 1$

9: **end if**

10: **return** FALSE $\triangleright 1$

11: **end while**

$$T(n) = 3n + 4$$

EXAMPLE: LINEAR SEARCH ALGORITHM

- **Best Case:** 'x' is located in the first position of the array A

$$\begin{aligned}T(n) &= 1 + 1 + 1 + 1 + 1 \\&= 5 \\&= O(1)\end{aligned}$$

- **Worst Case:** 'x' is located in last location of the array or is not there at all

$$\begin{aligned}T(n) &= 1 + (n + 1) + n + (n - 1) + 1 + 1 + 1 \\&= 3n + 4 \\&= O(n)\end{aligned}$$

- **Average Case:** Assume that it is equally likely for ‘ x ’ to appear at any position in array A ,. Accordingly, the number of comparisons can be any of the numbers $1, 2, 3, \dots, n$, and each number occurs with probability $p = 1/n$.

$$\begin{aligned}T(n) &= 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + \dots + n \cdot \frac{1}{n} \\&= (1 + 2 + 3 + \dots + n) \cdot \frac{1}{n} \\&= \frac{n(n+1)}{2} \cdot \frac{1}{n} \\&= \frac{n+1}{2} \\&= O(n)\end{aligned}$$

This agrees with our intuitive feeling that the average number of comparisons needed to find the location of ‘ x ’ is approximately equal to half the number of elements in the list A

EXAMPLE: ITERATIVE BINARY SEARCH

```
1: while  $low \leq high$  do  
2:    $mid = (low + high)/2$   
3:   if  $(x == arr[mid])$  then  
4:     return  $mid$   
5:   else if  $(x > arr[mid])$  then  
6:      $low = mid + 1$   
7:   else  
8:      $high = mid - 1$   
9:   end if  
10: end while
```

EXAMPLE: RECURSIVE BINARY SEARCH

```
1: function BINARYSEARCH(arr, x, low, high)
2:   if low > high then
3:     return False
4:   else
5:     mid = (low + high)/2
6:     if x == arr[mid] then
7:       return mid
8:     else if x > arr[mid] then
9:       return binarySearch(arr, x, mid + 1, high)
10:    else
11:      return binarySearch(arr, x, low, mid - 1)
12:    end if
13:  end if
14: end function
```

OPERATION COUNT EXAMPLE # 1

```
int x = 1, y = 10, z = 20;
```

```
for (int x = 1; x < N; x = x + 4)
```

```
    for (int y = 1; y < M; y = y * 4)
```

```
        Task a;
```

$$c_1 \cdot 1$$

$$c_2 \cdot (\lceil \frac{N-1}{4} \rceil + 1) \approx c_2 \frac{N}{4}$$

$$c_3 \cdot (\lceil \frac{N-1}{4} \rceil) \cdot (\lceil \log_4 M \rceil + 1) \approx \frac{N}{4} \log_4 M$$

$$c_4 \cdot (\lceil \frac{N-1}{4} \rceil) \cdot (\lceil \log_4 M \rceil) \approx \frac{N}{4} \log_4 M$$

$$\begin{aligned} T(n) &= c_1 + c_2 \frac{N}{4} + c_3 \frac{N}{4} \log_4 M + c_4 \frac{N}{4} \log_4 M \\ &= O(N \log_4 M) \end{aligned}$$

OPERATION COUNT EXAMPLE # 2

```
for (i = 0; i <= N; i = i + 2){  
    for (j = 1; j <= 2 * i; j = j + 2){  
        a = a * 2 - 3;  
    }  
}
```

$$\lfloor \frac{N}{2} \rfloor + 2 \approx \frac{N}{2}$$

$$(\lfloor \frac{N}{2} \rfloor + 1)(\frac{N}{2} + 1) = \frac{N+2}{2} \cdot \frac{N+2}{2} \approx N + \frac{N^2}{4}$$

$$(\lfloor \frac{N}{2} \rfloor + 1)(\frac{N}{2}) \approx \frac{N^2}{4}$$

$$T(n) = \frac{N}{2} + N + \frac{N^2}{4} + \frac{N^2}{4} = O(N^2)$$

```

for (int x = 1; x < N; x = x + 2)
{
    for (int y = 1; y < M; y = y * 3)
    {
        a = a + b;
    }
    b = a + b;
}

```

$$c_1 \cdot (\lfloor \frac{N}{2} \rfloor + 1) \approx \frac{N}{2}$$

$$c_2 \cdot \lfloor \frac{N}{2} \rfloor \cdot (\lceil \log_3 M \rceil + 1) \approx \frac{N}{2} \log_3 M$$

$$c_3 \cdot \lfloor \frac{N}{2} \rfloor \cdot \lceil \log_3 M \rceil \approx \frac{N}{2} \log_3 M$$

$$c_4 \cdot \frac{N}{2}$$

$$\begin{aligned}
T(n) &= c_1(\frac{N}{2}) + c_2 \frac{N}{2} (\log_3 M) + c_3 \frac{N}{2} (\log_3 M) + c_4 \frac{N}{2} \\
&= O(N \log_3 M)
\end{aligned}$$